

Universität Freiburg – Institut für Physik

Applied Theoretical Physics

Prof. Dr. Michael Moseler

Freiburg, 19.05.2026

Due: 02.06.2026 (Tuesday)

Computational Physics: Materials Science - Exercise 2, SS 2026

Email: andreas.doell@physik.uni-freiburg.de, severin.sylla@iwm.fraunhofer.de

Tutorials: Mr. Andreas Doell

Problem 4. Molecular dynamics of a sodium cluster

In this exercise, we simulate a cluster of sodium atoms interacting with the Morse potential. We will look at the cluster at different temperatures to identify when the cluster melts.

Implementation This exercise is a bit challenging, as you will have to deal with more than one particle. Additionally, you will need to implement lots of different new features. Please follow the guidelines to make sure your code works before moving onto the next step. Make sure that you understand what units you are using, as AI is very bad when with units.

We will work with the Morse potential in a slightly different form, see equation (1)

$$V(r) = D \left(e^{-2\alpha(r-r_0)} - 2e^{-\alpha(r-r_0)} \right), \quad (1)$$

with parameters given later. For now you do not need specific parameters. First we will learn how periodic boundaries work.

- (a) To study periodic boundary conditions, we create a "one-dimensional box" of length L with two particles inside it. Place one particle at the origin. Now place another particle at some distance and displace it step by step in small increments. When the second particle leaves the box at $r = \pm L/2$, shift it by L such that its *image* re-enters the box from the other side.
- (b) Calculate the so-called *minimum image* distance $|x_2 - x_1|_{min}$ which is the shortest distance either between the particles themselves or between a particle and the closest image of the other particle.
- (c) Let the particles interact via the Morse potential where the distance r is obeying the *minimum image* convention and calculate the potential energy. The parameters of the Morse potential do not matter in this context.
- (d) Plot the Morse potential using the minimum image distance between the particles and make sure it matches your expectations.
- (e) When particles are located very far from each other, their pairwise interaction approximately vanishes and can thus be neglected to improve computational efficiency. This motivates the usage of a cutoff radius r_{cut} . Atoms which are outside this radius do not interact with each other. However, just putting the potential to zero at this cutoff can lead to unphysical behaviour. To make sure that there are no discontinuities in the potential, shift the potential by its value at the cutoff distance: $V(r) = V_M(r) - V(r = r_{cut})$. The cutoff r_{cut} should be at least twice r_0 . Convince yourself that the potential looks as expected.

Now we can move onto the next step where we build up our MD simulation. First we need to initialize our system:

- (a) Create a 3D box where you can put in $N=27$ sodium atoms. Initialize your system in a box with lengths such that you obtain a density of 0.5 atoms/nm^3
- (b) Assign random initial velocity components to your system such that the initial temperature of the system is T_0 . For that, follow this protocol:
 - Assign initial velocity components between $(-1, 1)$
 - Shift all velocities by the mean to cancel the linear moment of the system such that $\sum_{i=1}^N v_{x0}^i = \sum_{i=1}^N v_{y0}^i = 0$
 - Rescale the velocities such that you obtain the target temperature of 100K using the equipartition theorem. Make sure that this *instantaneous temperature* matches your target temperature.
- (c) Visualize your system in ovito.

Now we can move to the dynamics part of our simulation. Use the following parameters and make sure to convert positions, velocities, forces and temperature accordingly to match units. For the Morse potential parameters, take the values for sodium from this source here: <https://journals.aps.org/pr/pdf/10.1103/PhysRev.114.687>. Choose $\Delta t=1$ (in units of fs) and simulate for at least 10000 steps. You can also convert these values into units of your liking if you desire to do so. In the end it does not matter what units you choose, you just need to be consistent!

- (a) The force $\mathbf{F} = -\nabla V(r)$ between two atoms is calculated via the Morse potential $V(r)$ obeying the minimum image convention. Make sure that only atoms i and j interact with each other with $i \neq j$ and that you only calculate the interaction between atoms which are inside the cutoff radius $r_{ij} < r_{cut}$. For each timestep calculate the corresponding forces of all atoms acting on each other and update the positions and velocities according to the Velocity Verlet algorithm. Save your positions every 100 steps in a file which can be visualized by ovito.
- (b) At every step plot the temperature $T(t)$. You will see that the temperature is not constant. Later, in your report discuss the different way of thermostating which you have seen in the lecture and what thermostat would result in the correct sampling of an NVT ensemble.
- (c) Implement a simple thermostat. Every 100 steps rescale the velocities such that the instantaneous temperature matches your target temperature again. Plot the temperature $T(t)$ and ensure that this works.
- (d) For proper MD we need a more sophisticated thermostat. For that we use the Berendsen thermostat. The idea of this thermostat is very simple: Instead of rescaling the temperature at some frequency, which is unphysical, we scale the velocities at every step with some factor λ : $\mathbf{v}_i^{new} = \lambda \mathbf{v}_i$ with

$$\lambda = \sqrt{1 + \frac{\Delta t}{\tau_T} \left(\frac{T_0}{T} - 1 \right)}, \quad (2)$$

where λ relates the instantaneous temperature T to the target temperature T_0 with a coupling constant τ_T and with timestep Δt , see equation (2). We use $\tau_T=100\Delta t$ in this exercise. Validate the thermostat by calculating the average temperature of the system, after some initial equilibration time.

If your code is too slow, make sure that you vectorize as much as you can and make use of the module **numba** which gives you access to the decorator **@njit**. This will allow

you later to go to larger amount of particles as your double loop over for the force scales as $\mathcal{O}(N^2)$. Try it out for yourself and see what happens if you go to $N=125$ particles! Additionally, you can go up to 50000 steps.

Analysis

- Obtain a working simulation and play around with the coupling constant of the Berendsen thermostat. What happens if $\tau_T = \Delta t$? Try to increase the number of atoms to over 100. Can you do it in a reasonable amount of time? Can you time your code for different N and different amount of steps?
- Visualize your simulation in Ovito at T=100K. What do you see? What do you see when you increase the temperature? What happens at T=300K and T=700K?
- What happens if you choose copper instead of sodium?

If you have issues with your code, please say something in your report. Try to check for consistent units. Often the relations between temperature and velocity can give problems. Explain what approaches you took to debug your code.

What to submit

Hand in a zip file on ILIAS containing the following files:

- your Python source code
- a short report in PDF format with your main results

A good submission contains:

- A detailed description of your code and the goal of the exercise.
- Make sure that you include the plots for validation and explain why it works and how it works.
- Finish up your submission with illustrative ovito plots of your cluster at different temperatures which can demonstrate the melting.